

Nombre:

Carné:



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

Examen Diciembre'24 C++

C++
PARA INGENIEROS



Informática II

Prof. Paul Bustamante / Jesús Paredes

Índice

1. INTRODUCCIÓN	1
2. GESTIÓN DE PRÉSTAMOS CON ESTRUCTURAS (2.0 PTS.).....	1
3. GESTIÓN DE SALAS DE CINE CON POO (2.0 PTS.).....	3
4. GESTIÓN DE UN BANCO CON HERENCIA (3.5 PTS).....	6

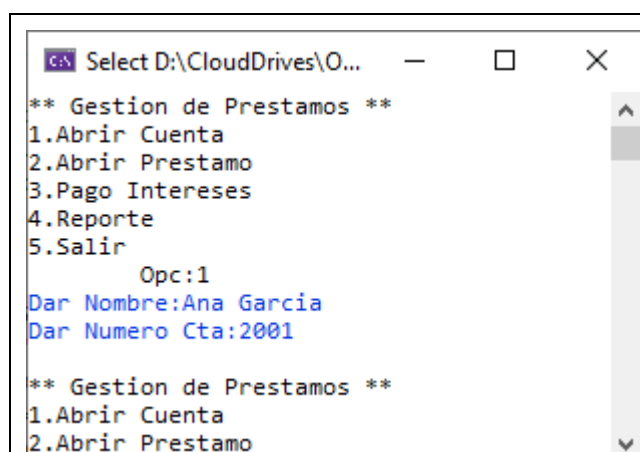
1. Introducción

Recuerde que debe dejar los ejercicios en la carpeta “**C:\Temp\ExamInfor**”. Si no los deja allí, no se le podrá corregir el examen. Trate de realizar primero el ejercicio que crea que es más sencillo y deje el complicado para el final.

2. Gestión de préstamos con estructuras (2.0 Pts.)

A Ud. le han pedido que haga un programa para la gestión de préstamos de una entidad bancaria mediante estructuras del C++. Dicho programa deberá tener las siguientes opciones:

- **Opcion1 (0.4 Pts.):** Abrir Cuenta: Para poder abrir o crear un préstamo, primeramente, debe crea una cuenta, con el Nombre y el número de cuenta. Ver **Figura 1**.
- **Opcion2 (0.50 Pts.):** Abrir o crear Préstamos: Para poder crear un préstamo, debe hacer un listado de las cuentas que hay en la entidad, para luego seleccionar la cuenta asociada al préstamo. Luego deberá pedir la **fecha** del préstamo y la **cantidad** (Euros). Ver **Figura 2**.
- **Opcion3 (0.70 Pts.):** Pago de intereses: Primero deberá pedir la **fecha actual** para calcular el número de días transcurridos, y por consiguiente el monto a pagar. Luego hará un listado de todos los préstamos creados con los datos del préstamo y el número de días y monto a pagar. Ver **Figura 3**.
- **Opcion4 (0.20 Pts.):** Grabar todo a un fichero de texto (cuentas y prestamos). Debe pedir el nombre del fichero a grabar (pe. Datos.txt).



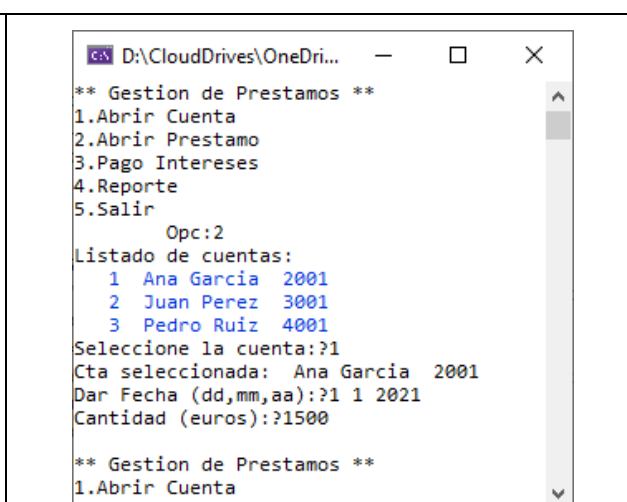
```

** Gestion de Prestamos **
1.Abrir Cuenta
2.Abrir Prestamo
3.Pago Intereses
4.Reporte
5.Salir

    Opc:1
Dar Nombre:Ana Garcia
Dar Numero Cta:2001

** Gestion de Prestamos **
1.Abrir Cuenta
2.Abrir Prestamo
    
```

Figura 1 Abrir o crear cuenta



```

** Gestion de Prestamos **
1.Abrir Cuenta
2.Abrir Prestamo
3.Pago Intereses
4.Reporte
5.Salir

    Opc:2
Listado de cuentas:
 1 Ana Garcia 2001
 2 Juan Perez 3001
 3 Pedro Ruiz 4001
Seleccione la cuenta:?1
Cta seleccionada: Ana Garcia 2001
Dar Fecha (dd,mm,aa):?1 1 2021
Cantidad (euros):?1500

** Gestion de Prestamos **
1.Abrir Cuenta
    
```

Figura 2 Abrir o crear préstamo

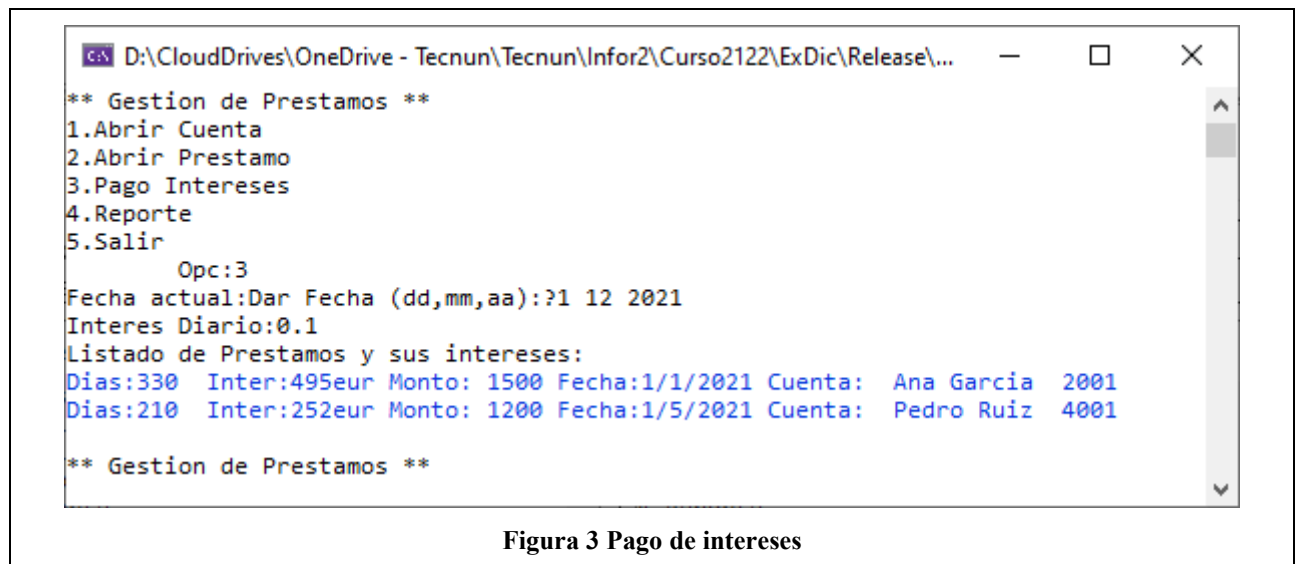


Figura 3 Pago de intereses

Programa:

Para la realización del programa, deberá crear las siguientes estructuras: **cuenta**, **prestamo** y **Fecha**.

<pre>struct cuenta { int numero; //pe 2001 char *name; //puntero para nombre };</pre>	<pre>struct prestamo { cuenta cta; //cuenta asociada Fecha faper; //fecha apertura double cantidad; //euros };</pre>
---	---

La estructura **cuenta** tiene un puntero para almacenar el nombre, lo cual significa que debe reservar memoria de forma dinámica, usando **new**.

Deber hacer una estructura **Fecha** con tres miembros enteros: **día**, **mes**, **año**.

En cuanto a **main**, a continuación, se da un esqueleto de cómo podría ser:

```
#define MAX 100 //num máximo de préstamos
#define INTERES 0.1 //interés diario en %.

void main()
{
    int np = 0;
    prestamo lstPres[MAX]; //array de préstamos
    int nc = 0;
    cuenta lstCuenta[MAX]; //array de cuentas

    int opc=0;
    while (1) {
        // mostrar el menu y hacer el resto del código
    }
}
```

Función principal (**main**) más optimización del código: **0.2** Pts.

3. Gestión de salas de cine con POO (2.0 Pts.)

Se le ha pedido realizar un programa para la gestión de unas salas de cine, utilizando la POO.

Para ello deberá crear dos clases: clase **hora** y clase **sala**.

```
class hora
{
private:
    int hh;        //hora
    int min;       //minutos
public:
    hora();
};

class sala
{
private:
    char asiento[20][30]; //max 20 filas y 30 asientos/fila: '0' ocupado
    int filas, cols;      //numero real de filas y columnas
    char pelicula[40];     //nombre de la pelicula
    double precio;        //precio
    hora h;               //hora de la sesion
public:
    sala();
    void prt();           //imprimir la ocupación de la sala
    void comprar();       //comprar entradas a la sala
    void devolver();      //devolver entradas (una a una)
    double recaudado();   //mostrar la ocupación y lo recaudado
};
```

Al inicio del programa deberá crear las salas (debe pedir el número de salas a crear) que va a tener, dando todos los parámetros a cada sala: número de **filas**, número de **asientos por fila**, nombre de la **película**, **hora** de la sesión y el **precio** (vea la siguiente figura):

```
*** Gestion de Cines ***

Dar numero de salas:?2
----- Dar Datos de la Sala 1 -----
Numero de filas:?8
Numero de asientos/fila:?10
Pelicula:?Venom
Hora:?20 30
Precio:?8.5
----- Dar Datos de la Sala 2 -----
Numero de filas:?10
Numero de asientos/fila:?15
Pelicula:?Gladiador
Hora:?19 30
Precio:?9.5

**** Menu Cines Max ****
1.Comprar Entradas
2.Devolver Entradas
3.Ver salas y dinero recaudado
4.Salir
Opc:?
```

Luego el programa mostrará un menú con 4 opciones:

- **Opción 1 (0.5 pts): Comprar** entradas. En esta opción deberá listar las salas y la película. El usuario deberá seleccionar la sala y proceder a la compra de entradas (esto lo hará mediante la función **comprar()** de la clase **sala**). Deberá mostrar el “mapa” de asientos, donde ‘0’ es que está libre y ‘1’ es que ya está ocupado (usando la matriz **char asientos[][]** de la clase

sala). Si el asiento ya está ocupado, deberá mostrar un mensaje. Vea la siguiente figura de cómo debería ser la compra:

```

E:\Tecnun\ExDic24\x64\Debu! X + - □ X
**** Menu Cines Max ****
1.Comprar Entradas
2.Devolver Entradas
3.Ver salas y dinero recaudado
4.Salir
Opc:~1
Listado de Peliculas por sala:
Sala:1 Pelicula:Venom
Sala:2 Pelicula:Gladiador
Que sala desea comprar:~1
Pelicula:Venom
Hora :20:30
  1  2  3  4  5  6  7  8  9 10
1  0  0  0  0  0  0  0  0  0  0
2  0  0  0  0  0  0  0  0  0  0
3  0  0  0  0  0  0  0  0  0  0
4  0  0  0  0  0  0  0  0  0  0
5  0  0  0  0  0  0  0  0  0  0
6  0  0  0  0  0  0  0  0  0  0
7  0  0  0  0  0  0  0  0  0  0
8  0  0  0  0  0  0  0  0  0  0
Numero de entradas:~2
Dar Fila Asiento:~3 4
Dar Fila Asiento:~3 5
A pagar:17 euros

**** Menu Cines Max ****
1.Comprar Entradas

```

- **Opción 2 (0.5 pts): Devolver** entradas. Mediante esta opción deberá poder devolver la entrada comprada, para lo cual primero deberá pedir la sala y luego la fila y el asiento. Deberá “liberar” el asiento asignado en la matriz asientos. Esta opción debe hacerla en la función **devolver()**.

```

E:\Tecnun\ExDic24\x64 X + - □ X
**** Menu Cines Max ****
1.Comprar Entradas
2.Devolver Entradas
3.Ver salas y dinero recaudado
4.Salir
Opc:~2
Listado de salas:
Sala:1 Pelicula:Venom
Sala:2 Pelicula:Gladiador
De que sala desea devolver la entrada:~1
Dar Fila Asiento:~3
4
Devuelto: 8.5 euros

**** Menu Cines Max ****
1.Comprar Entradas
2.Devolver Entradas
3.Ver salas y dinero recaudado
4.Salir
Opc:~|

```

- **Opción 3 (0.6 pts): Ver salas y dinero recaudado.** Esta opción permitirá ver el “mapa de ocupación” de cada una de las salas, así como el dinero recaudado por sala y el total. Debe usar la función **prt()** para mostrar la ocupación de la sala.

```

E:\Tecnun\ExDic24\x64\Debu X + - □ ×
**** Menu Cines Max ****
1.Comprar Entradas
2.Devolver Entradas
3.Ver salas y dinero recaudado
4.Salir
Opc:3
Dinero recaudado:
Sala: 1
Pelicula:Venom
Hora :20:30
  1 2 3 4 5 6 7 8 9 10
1 0 0 0 0 0 0 0 0 0 0
2 0 0 0 0 0 0 0 0 0 0
3 0 0 0 1 1 0 0 0 0 0
4 0 0 0 0 0 0 0 0 0 0
5 0 0 0 0 0 0 0 0 0 0
6 0 0 0 0 0 0 0 0 0 0
7 0 0 0 0 0 0 0 0 0 0
8 0 0 0 0 0 0 0 0 0 0
Euros:17
Sala: 2
Pelicula:Gladiator
Hora :19:30
  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
2 0 0 0 0 0 0 0 0 0 0 0 0 0 0
3 0 0 0 0 0 0 0 0 0 0 0 0 0 0
4 0 0 0 0 0 0 0 1 1 0 0 0 0 0
5 0 0 0 0 0 0 0 0 0 0 0 0 0 0
6 0 0 0 0 0 0 0 0 0 0 0 0 0 0
7 0 0 0 0 0 0 0 0 0 0 0 0 0 0
8 0 0 0 0 0 0 0 0 0 0 0 0 0 0
9 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Euros:19
Total recaudado:36 Euros
**** Menu Cines Max ****

```

La función principal **main (0.4Pts)** pedirá los datos iniciales para las salas y también tendrá un menú para elegir las opciones que se han planteado.

La estructura de **main** puede ser la siguiente:

```

//incluir las cabeceras
void main()
{
    sala* lst    //vector lista de salas
    //agregar más variables

    //Crear las salas y sus datos

    int opc;
    while(true){
        opc=Menu();
        //opcion1: comprar entradas
        //opcion2: devolver entradas
        //opcion3: Ver listado de salas y lo recaudado
        //opcion4: terminar el programa

    }

    cout << "Fin"<<endl ;
}

```

4. Gestión de un banco con herencia (3.5 Pts)

Se le ha encargado realizar un programa para gestionar cuentas bancarias. Se trata de una versión de desarrollo, por lo que no se le pedirá realizar ningún menú. Para ello deberá utilizar 3 clases (cuenta, renta_mixta y renta_fija) siguiendo la jerarquía indicada en la siguiente figura.

<pre>class cuenta { private: static double interes_base; char titular[20]; protected: double *saldo; int cont; // añade el código necesario };</pre>	 <pre> graph TD Cuenta --> RentaMixta[Renta mixta] RentaMixta --> RentaFija[Renta fija]</pre>
<pre>class renta_mixta // derivada de cuenta { double interes_fijo; double prop_variable; // añade el código necesario };</pre>	<pre>class renta_fija // derivada de renta mixta { // añade el código necesario };</pre>

Los objetos de la **clase "cuenta"** tienen como variables privadas: un array `char "titular[20]"` que almacena el nombre del titular; y un `static double` llamado `"interes_base"` (cuyo valor, en tanto por uno, se define en el *main*) que se corresponde con el interés que recibe el titular por el mero hecho de tener saldo. Como variables protegidas cuenta con una variable `int "cont"` y un array de `double "saldo"`. En la variable `"saldo"` se almacenará la evolución del saldo de la cuenta según se vayan realizando movimientos, y la variable `"cont"` indica en qué posición de dicho array `"saldo"` se encuentra actualmente.

Los objetos de la **clase "renta_mixta"** heredan todas las variables y funciones de la clase `"cuenta"`, pero además cuentan con dos variables `double` privadas. La variable `"interes_fijo"` se corresponde con el rendimiento (en tanto por uno) al que se remunera el capital depositado a interés fijo. La variable `"prop_variable"` indica la proporción (en tanto por uno) de capital invertido en interés variable respecto del total.

Los objetos de la **clase "renta_fija"** heredan todas las funciones y variables de la clase `"renta_mixta"` y no tiene variables propias.

El objetivo de este ejercicio es desarrollar las funciones necesarias que se indican en los siguientes apartados. Para evaluar su funcionamiento se le proporcionan datos de la función *main* y el output por consola esperado.

A. (0.3Pts) Constructores y destructores

- Puede crear tantos constructores como desee.
- SIEMPRE que se cree un objeto de tipo `"cuenta"`, tipo `"renta_mixta"`, o tipo `"renta_fija"`, debe asignar una dimensión de 100 al array dinámico `"saldo"` (no es necesario que se la pida un valor al usuario) y debe inicializar el contador de movimientos a cero.
- Deberá ser capaz de asignar un saldo inicial a las variables de cada objeto que cree: puede hacerlo de la manera que crea más conveniente.
- La clase `"renta_fija"` es un caso particular de la clase `"renta_mixta"` donde la variable `"prop_variable"` vale siempre cero.
- No se olvide de los destructores y de liberar la memoria dinámica.

```
double cuenta::interes_base = 0.001;
void main()
{
    // Si lo desea puede asignar valores a estos objetos mediante
    // funciones, no es necesario que lo haga a través de constructores. Esto
    // es solo un ejemplo

    cuenta c1((char*)"Pedro",1000);
    renta_mixta m1((char*)"Maria", 1000, 0.2, 0.05);
    renta_fija f1((char*)"Paco", 1000, 0.05);

    // Sigue más adelante
```

B. (0.5Pts) Cree una función **pública bool Gasto(double x)** en las clases donde sea estrictamente necesaria.

- Esta función tiene como argumento de entrada una variable **double "x"** correspondiente al gasto que se desea ejecutar, y devuelve como variable de salida un **bool** que indica si la operación se ha podido ejecutar (**true**) o no (**false**).
- Cuando un objeto de la clase *"cuenta"* llame a esta función, deberá:
 - Comprobar que el valor *"x"* es positivo y comprobar si el saldo actual de la cuenta es suficiente.
 - En caso negativo, la función deberá devolver un **false**.
 - En caso afirmativo, el nuevo saldo deberá quedar reflejado en el vector *"saldo"*, por lo que el contador de operaciones (*"cont"*) se incrementa en uno. El nuevo saldo se corresponderá con el saldo anterior menos la cantidad *"x"* (argumento de entrada). Finalmente devolverá un **true**.
- Cuando objetos de clases derivadas (*"renta_mixta"* o *"renta_fija"*) llamen a esta función, no debe hacer nada, únicamente devolverá un **false**.

```
// Continuación main

cout << c1.Gasto(200) << endl;
cout << c1.Gasto(1000) << endl;

cout << m1.Gasto(200) << endl;
cout << f1.Gasto(200) << endl;

// Sigue más adelante
```

Output de consola



```
1
0
0
0
```

C. (0.2Pts) Cree una función **pública** llamada **"prt"** en las clases donde sea estrictamente necesaria.

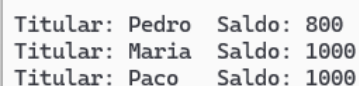
- No tiene argumentos de entrada.
- No tiene argumentos de salida.
- Muestra por pantalla el nombre del titular y el saldo actual.

```
// Continuación main

cout << endl;
c1.prt();
m1.prt();
f1.prt();

// Sigue más adelante
```

Output de consola



```
Titular: Pedro Saldo: 800
Titular: Maria Saldo: 1000
Titular: Paco Saldo: 1000
```

D. (0.5Pts) Cree una función **pública** llamada “**historial**” en las clases donde sea estrictamente necesaria.

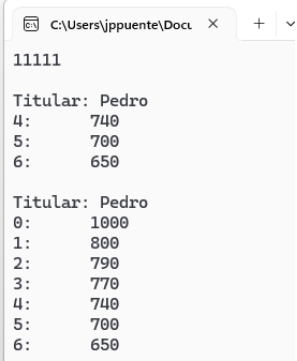
- Como argumento de entrada tiene una variable **int** “**num**” que indica el número de movimientos que desea mostrar.
- No tiene argumentos de salida.
- Muestra el nombre del titular y los “**num**” últimos movimientos (entre los que se incluye el saldo actual). Si “**num**” es mayor que la cantidad de movimientos guardados, se mostrarán todos los movimientos.

```
// Continuación main

cout << endl;
cout << c1.Gasto(10);
cout << c1.Gasto(20);
cout << c1.Gasto(30);
cout << c1.Gasto(40);
cout << c1.Gasto(50);
cout << endl << endl;
c1.historial(3);
cout << endl;
c1.historial(40);

// Sigue más adelante
```

Output de consola



```
11111

Titular: Pedro
4: 740
5: 700
6: 650

Titular: Pedro
0: 1000
1: 800
2: 790
3: 770
4: 740
5: 700
6: 650
```

E. (0.5Pts) Cree una función **pública** llamada “**transferencia**” en las clases donde sea estrictamente necesaria.

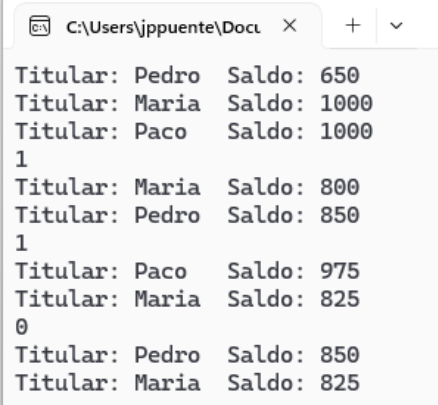
- Al igual que con la opción del apartado 2, esta función devolverá como argumento de salida una variable **bool** que determine si ha sido posible realizar la operación (**true**) o no (**false**).
- Esta función tendrá como argumentos de entrada la cuenta de destino y un **double** correspondiente a la cantidad a transferir.
- Tenga en cuenta que esta función debe comprobar si hay saldo en la cuenta de origen (se corresponde con el objeto que llama a la función), e incrementar el saldo de la de destino (el objeto que entra como argumento de entrada). El nuevo saldo de ambas cuentas deberá quedar en el registro de ambas cuentas, es decir, en el array “**saldo**” de cada una, de manera análoga a como se hace en la función “**Gasto**”.

```
// Continuación main

cout << endl;
c1.prt();
m1.prt();
f1.prt();
cout<<m1.transferencia(c1,200)<<endl;
m1.prt();
c1.prt();
cout<<f1.transferencia(m1,25)<<endl;
f1.prt();
m1.prt();
cout<<c1.transferencia(f1,5000)<<endl;
c1.prt();
m1.prt();

// Sigue más adelante
```

Output de consola



```
Titular: Pedro Saldo: 650
Titular: Maria Saldo: 1000
Titular: Paco Saldo: 1000
1
Titular: Maria Saldo: 800
Titular: Pedro Saldo: 850
1
Titular: Paco Saldo: 975
Titular: Maria Saldo: 825
0
Titular: Pedro Saldo: 850
Titular: Maria Saldo: 825
```

F. (0.5Pts) Deberá crear una función **PRIVADA** llamada **"get_intereses"** en las clases donde sea estrictamente necesaria.

- No tiene variables de entrada.
- Deberá devolver el interés como variable **double**:
 - El interés de una cuenta genérica (de la clase base) es el indicado en la variable interés base.
 - El interés de una renta mixta: el interés mixto se calcula como la suma ponderada del interés fijo y el interés variable. Por ejemplo, si el valor de la variable **"prop_variable"** es 0.2, entonces el interés se calcula como: $\text{interés_mixto} = 0.8 * \text{intereses-fijos} + 0.2 * \text{interés-variable}$.
 - El interés variable se establece como una variable aleatoria de valor entre - 0.5 y 1.5.
 - Los intereses-fijos se calculan como la suma de la variable **"interes_fijo"** y la variable **"interes_base"**.
 - El interés de una renta fija se calcula igual que el de la renta variable, pero en este caso el valor de la **"prop_variable"** es 0.

G. (0.3Pts) Deberá crear una función **pública** llamada **"Simular"** en la clase **"cuenta"** (solo en la clase **"cuenta"**).

- Como variable de entrada tiene una variable **int** que se corresponde con el número de años/meses que transcurren. Esta variable funciona como contador del número de veces que se pagarán los intereses.
- Esta función devuelve un array de **double** con la evolución que tendría el saldo de la cuenta. Esta unción NO debe modificar el **"saldo"** de la cuenta.
- Dentro de esta función se deberá llamar a la función **get_intereses**.

```
// Continuación main

cout << endl;
double* res = c1.simular(5);
for (int i = 0; i < 5; i++)
{
    cout << i << ":\t" << res[i] << endl;
}
cout << endl;
double* res1 = m1.simular(5);
for (int i = 0; i < 5; i++)
{
    cout << i << ":\t" << res1[i] << endl;
}
cout << endl;
double* res2 = f1.simular(5);
for (int i = 0; i < 5; i++)
{
    cout << i << ":\t" << res2[i] << endl;
}
// Añada código si es necesario
// Sigue más adelante
```

Output de consola

C:\Users\jppuente\Docu
X + ▾

0:	850
1:	850.85
2:	851.701
3:	852.553
4:	853.405
0:	825
1:	694.176
2:	779.277
3:	730.535
4:	909.64
0:	975
1:	1024.72
2:	1076.99
3:	1131.91
4:	1189.64

H. (0.4Pts) Cree una función **pública** llamada **"grabar"** perteneciente a la clase **"cuenta"**.

- No tiene variables de salida.
- Como variable de entrada tiene un objeto de la clase `ofstream`.
- Esta función escribirá en un fichero (utilizando el objeto de clase `ofstream` que entra como argumento de entrada), el número de movimientos de la cuenta, seguido de todos los movimientos de la cuenta.

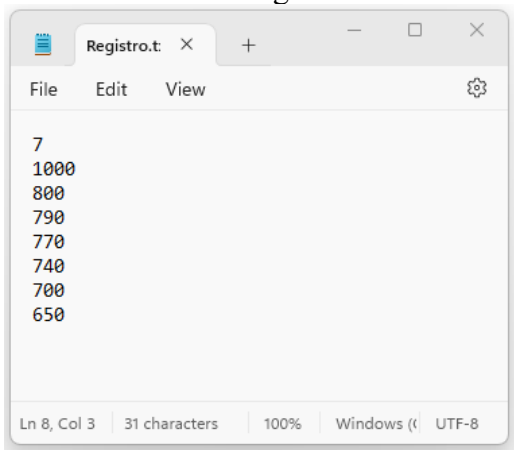
```
// Continuación main

// crear objeto de lectura de un fichero
llamado "Registro.txt"

c1.grabar(objeto_escritura);

//código necesario después

// Sigue más adelante
```



I. (0.3Pts) Cree una función `pública` llamada `"leer"` perteneciente a la clase `"cuenta"`.

- Como variable de salida devolverá un objeto de tipo cuenta.
- Como variable de entrada tendrá un objeto de la clase `ifstream`.
- Esta función lee en primer lugar un número, que le indica la cantidad de datos que tiene que leer. Posteriormente lee todos los saldos que almacenará en la variable saldo.

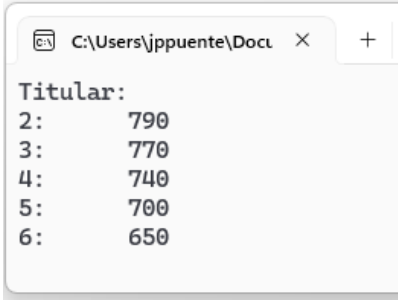
```
// Continuación main

// crear objeto de lectura de un fichero
llamado "Registro.txt"

cuenta c2;
c2.leer(objeto_lectura);
cout << endl;
c2.historial(5);

//código necesario después

// Sigue más adelante
```



Se **penalizará** cada `virtual` que no sea estrictamente necesario para que funcione el código del main propuesto.

Se **penalizará** si las funciones (p.ej. `Gasto`, `prt()`, `historial()`,...) se definen en más clases que aquellas en las que sea estrictamente necesario.

¡Suerte!!